CS432: Databases

Practical Database Design Methodology

Instructor
Yogesh K. Meena

Lecture no.
14

CSE, IIT Gandhinagar
March 13, 2026

Overview

- Information System Life Cycle
- Phases of Database Design
- UML Diagrams
  - Rational Rose
  - Other tools
- Design Tools

Organizational Context for Database Systems

- Consolidation and integration of data across organization
- Maintenance of complex data
- Simplicity of developing new applications
- **Data independence**
  - Protecting application programs from changes in the underlying logical organization and in the physical access paths and storage structures
- **External Schemas**
  - Allow the same data to be used for multiple applications with each application having its own view of the data

Information System

- Information System includes all resources involved in the collection, management, use and dissemination of the information resources of the organization
- We consider two systems life cycles:
  - **Macro Life Cycle**
    - Information System Life Cycle
  - **Micro Life Cycle**
    - Database System Life Cycle

Phases of Information System Life Cycle

- **Feasibility Analysis**
  - Analyzing potential application areas
  - Identifying the economics of information gathering and dissemination
  - Performing cost benefit studies
  - Setting up priorities among applications
- **Requirement Collection and Analysis**
  - Detailed Requirements Collection
  - Interaction with Users
- **Design**
  - Design of Database System
  - Design of programs that use and process the database

Phases of Information System Life Cycle

- **Implementation**
  - Information system is implemented
  - Database is loaded & its transactions are implemented and tested
- **Validation and Acceptance Testing**
  - Testing against user's requirements
  - Testing against performance criteria
- **Deployment, Operation and Maintenance**
  - Data conversion
  - Training
  - System maintenance
  - Performance monitoring
  - Database tuning

Database System Life Cycle

- **System definition**
  - Defining scope of database system, its users and Applications
- **Database Design**
  - Logical and physical design of the database system on the chosen DBMS
- **Database implementation**
  - Specifying conceptual, external and internal database definitions
  - Creating empty database files
  - Implementing software applications

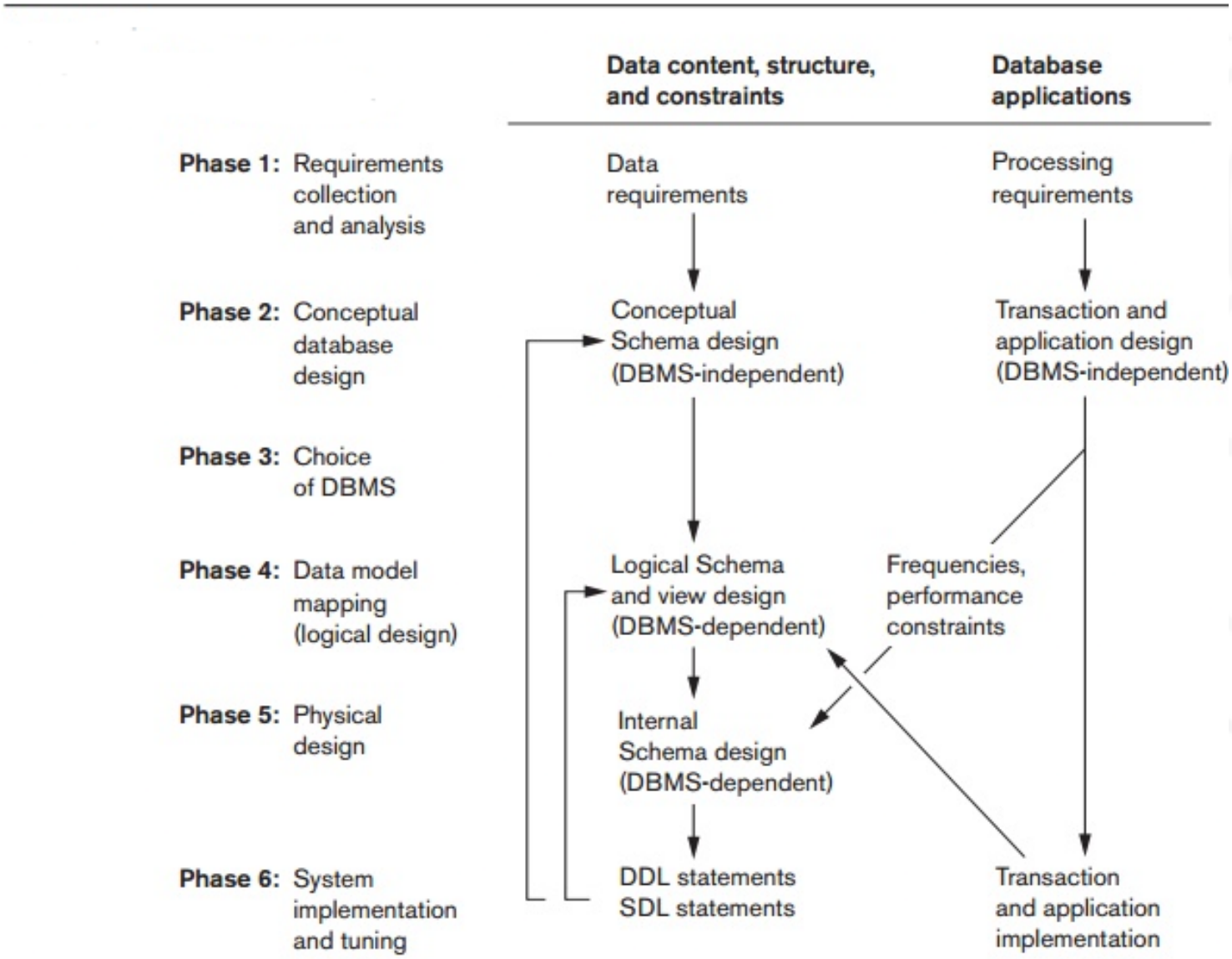
Database System Life Cycle (contd.)

- **Loading or data conversion**: Populating the database
- **Application conversion**: Converting applications to the new system
- **Testing and validation**
- **Operation**: Running the new system
- **Monitoring and maintenance**
  - System maintenance and performance monitoring

Database Design Process

- **Problem**
  - Design the logical and physical structure of one or more databases to accommodate the information needs of the users in an organization for a defined set of applications
- **Goals**
  - Satisfy the content requirements
  - Provide easy structuring of information
  - Support processing requirements and performance objectives

Database Design and Implementation Phases



Phase 1: Requirements Collection and Analysis

- **Requirements Collection and Analysis**
  - Identifying Users
  - Interacting with users to gather requirements
  - Time consuming BUT very important
    - Very expensive to fix requirements error
- **Conceptual Database Design**
  - Satisfy the content requirements
  - Provide easy structuring of information
  - Support processing requirements and performance objectives

Phase 2: Conceptual Database Design

- Produce a conceptual schema for the database that is independent of a specific DBMS
- Involves two parallel activities:
  - Conceptual Schema Design
  - Transaction and Application Design
- Conceptual Schema Design
  - Goal
    - Complete understanding of the database structure, semantics, interrelationships and constraints
  - Serves as a stable description of the database contents
  - Good understanding crucial for the users and designers
  - Diagrammatic description serves as an excellent communication tool

Desired Characteristics – Conceptual Model

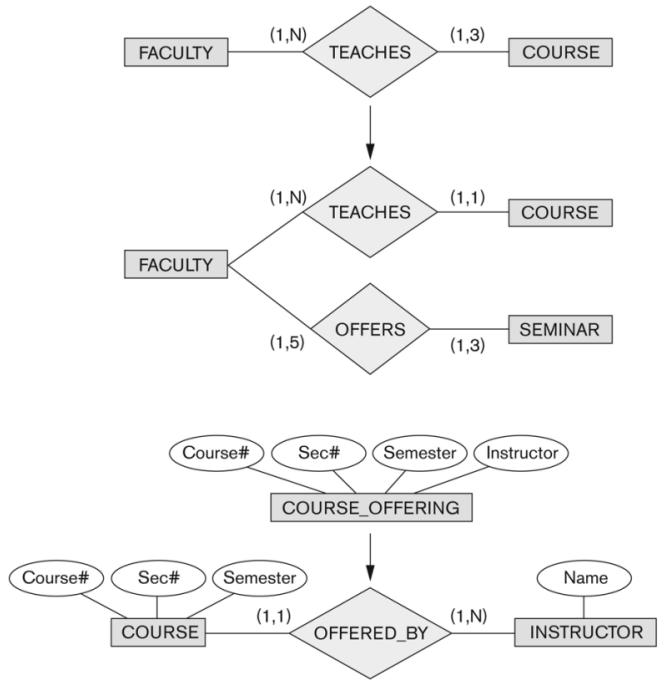
- Expressiveness
  - Able to distinguish different types of data, relationships and constraints
- Simplicity and Understandability
  - Easy to understand
- Minimality
  - Small number of distinct basic concepts
- Diagrammatic Representation
  - Diagrammatic notation for representing conceptual schema
- Formality
  - Formal unambiguous specification of data

Approaches to Conceptual Schema Design

- **Centralized Schema Design Approach**
  - Also known as one-shot approach
  - Requirements of different applications and user groups are merged into a single set of requirements and a single schema is designed
  - Time consuming, places the burden on DBA to reconcile conflicts
- **View Integration Approach**
  - Schema is designed for each user group or application
  - These schemas are then merged into a global conceptual schema during the view integration phase
  - More practical

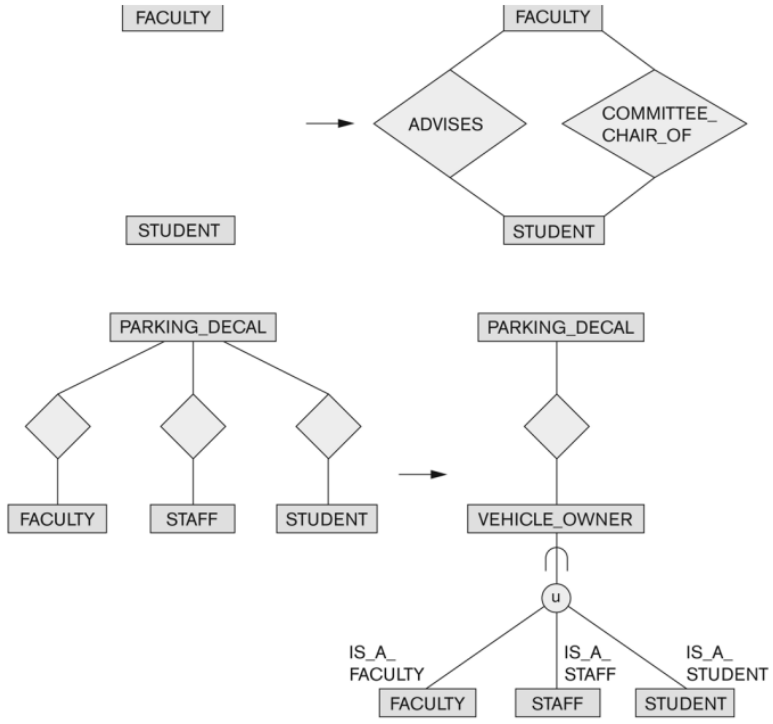
Strategies for Schema Design

- **Top Down Strategy**
  - Start with a schema containing high level abstractions and then apply successive top down refinements
  - Top: generating a new entity type
  - Bottom: decomposing an entity type into two entity types and relationship type



Strategies for Schema Design

- **Bottom-Up Strategy**
  - Start with a schema containing basic abstractions and then combine or add to these abstractions
  - Top: discovering and adding a new relationship
  - Bottom: discovering a new category (union type) and relating it



Strategies for Schema Design

- **Inside-Out Strategy**
  - Start with central set of concepts
  - Spread outward by considering new concepts in the vicinity of existing ones
- **Mixed Strategy**
  - Use a combination of top-down and bottom-up strategies

Schema Integration

Identifying correspondence and conflict among different schemas

- **Naming Conflicts**
  - Synonyms: The same concept but different names
    - e.g. entity types CUSTOMER and CLIENT
  - Homonyms: Different concepts but same name
    - e.g. entity type PART as computer parts and furniture parts

Schema Integration

Identifying correspondence and conflict among different schemas

- **Naming Conflicts**
  - Synonyms: The same concept but different names
    - e.g. entity types CUSTOMER and CLIENT
  - Homonyms: Different concepts but same name
    - e.g. entity type PART as computer parts and furniture parts
- **Type Conflicts:** Representing the same concept by different modeling constructs
  - e.g. DEPARTMENT may be an entity type and an attribute

Schema Integration

Identifying correspondence and conflict among different schemas

- **Naming Conflicts**
  - Synonyms: The same concept but different names
    - e.g. entity types CUSTOMER and CLIENT
  - Homonyms: Different concepts but same name
    - e.g. entity type PART as computer parts and furniture parts
- **Type Conflicts:** Representing the same concept by different modeling constructs
  - e.g. DEPARTMENT may be an entity type and an attribute
- **Domain Conflicts:** Attribute has different domains
  - Also known as value set conflicts
  - e.g. SSN as an integer and as a character string

Schema Integration

Identifying correspondence and conflict among different schemas

- **Naming Conflicts**
  - Synonyms: The same concept but different names
    - e.g. entity types CUSTOMER and CLIENT
  - Homonyms: Different concepts but same name
    - e.g. entity type PART as computer parts and furniture parts
- **Type Conflicts:** Representing the same concept by different modeling constructs
  - e.g. DEPARTMENT may be an entity type and an attribute
- **Domain Conflicts:** Attribute has different domains
  - Also known as value set conflicts
  - e.g. SSN as an integer and as a character string
- **Conflict among constraints:** Two schemas impose different constraints
  - e.g. different key of an entity type in different schemas

Schema Integration (contd.)

- Modifying views to conform to one another
- Merging of views
  - Merging Schemas to create a global schema
  - Specifying mappings between views and global schema
    - Time consuming and difficult
- Restructuring
  - Simplifying and restructuring to remove any redundancies

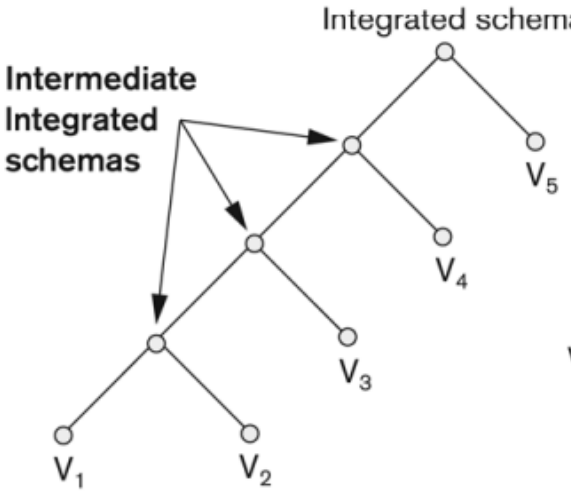
View Integration Strategies - Approaches

- **Binary Ladder Integration**
  - Two similar schemas are integrated first
  - Resulting schema is integrated with another schema
- **N-ary Integration**
  - All views are integrated in one procedure
  - Requires computerized tools for large designs

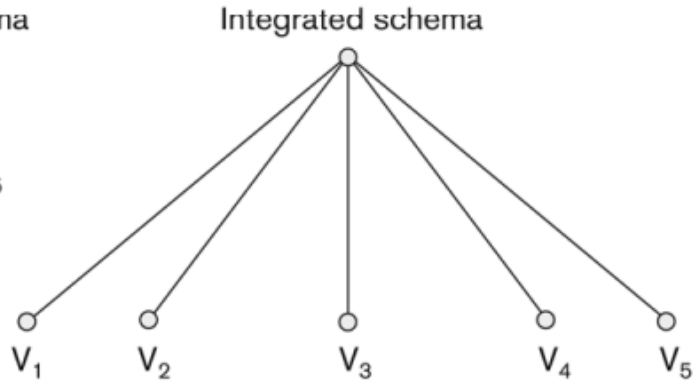
View Integration Strategies - More Approaches

- **Binary Balanced Strategy**
  - Pairs of schemas are integrated first
  - Resulting schemas are then paired for further integration
  - Process repeated until final global schema
- **Mixed Strategy**
  - Schemas partitioned into groups based on similarity
  - Each group integrated separately
  - Process repeated until final global schema

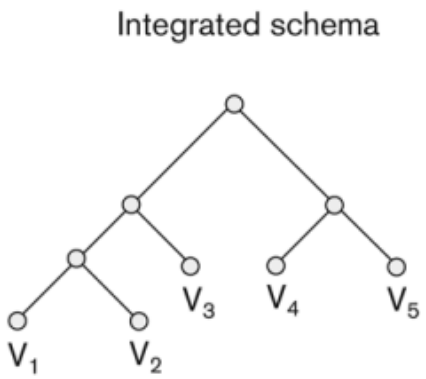
View Integration Strategies (contd.)



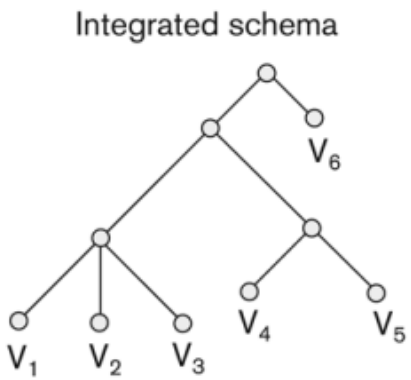
Binary ladder integration



N-ary integration



Binary balanced integration



Mixed integration

Transaction Design

- Design characteristics of known database transactions in a DBMS
- **Types of Transactions**
  1. **Retrieval Transactions**: Used to retrieve data
  2. **Update Transactions**: Update data
  3. **Mixed Transactions**: Combination of update and retrieval
- **Techniques for Specifying Transactions**
  - Input/output
  - Functional Behaviour

Phase 3: Choice of DBMS

Many factors to consider:

- **Technical Factors**
  - Type of DBMS: Relational, object-relational, object etc.
  - Storage Structures
  - Architectural options
- **Economic Factors**
  - Acquisition, maintenance, training and operating costs
  - Database creation and conversion cost
- **Organizational Factors**
  - Organizational philosophy (Relational or Object Oriented)
  - Vendor Preference
  - Familiarity of staff with the system
  - Availability of vendor services

Phase 4: Logical Database Design

- Transform the Schema from high-level data model into the data model of the selected DBMS
- Design of external schemas for specific applications
- **Two stages**
  - System-independent mapping
  - DBMS independent mapping
- Tailoring the schemas to a specific DBMS
  - Adjusting the schemas obtained in step 1 to conform to the specific implementation features of the data model used in the selected DBMS
- **Result**: DDL statements in the language of the chosen DBMS

Phase 5: Physical Database Design

- Design the specifications for the stored database in terms of physical storage structures, record placements and indexes
- **Design Criteria**
  - **Response Time**: Elapsed Time between submitting a database transaction for execution and receiving a response
  - **Space Utilization**: Storage space used by database files and their access path structures
  - **Transaction throughput**: Average number of transactions per minute (must be measured under peak conditions)
- **Result**: Initial determination of storage structures and access paths for database files

Phase 6: Database Implementation and Tuning

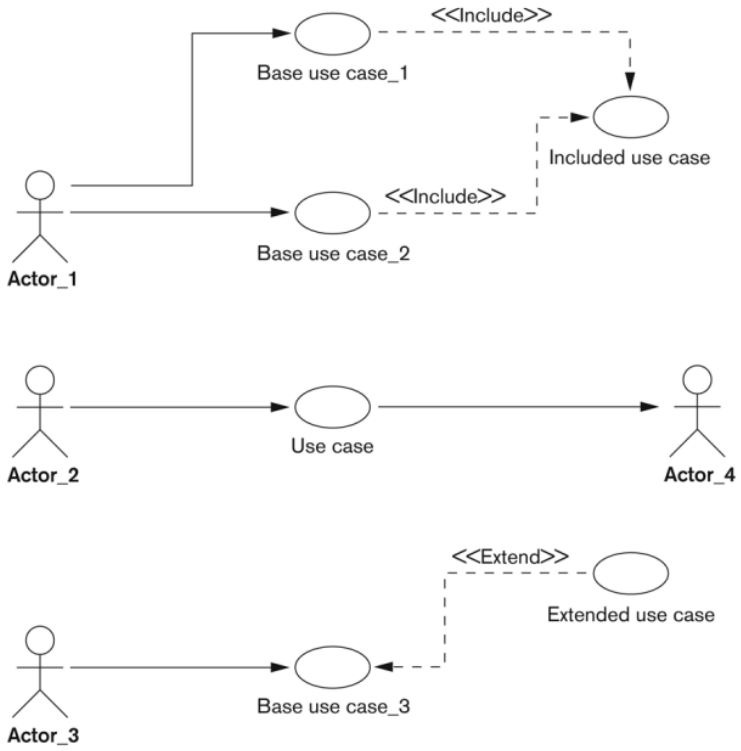
- During this phase database and application programs are implemented, tested and deployed
- **Database Tuning**
  - System and Performance Monitoring
  - Data indexing
  - Reorganization
  - Tuning is a continuous process

UML Diagrams

- **Class Diagrams**
  - Capture the static structure of the system
  - Represent classes, Interfaces, dependencies, generalizations and other relationships
- **Object Diagrams**
  - Show a set of objects and their relationships
- **Component Diagrams**
  - Show the organizations and dependencies among software components
- **Deployment Diagrams**
  - Represent the distribution of components across the hardware topology

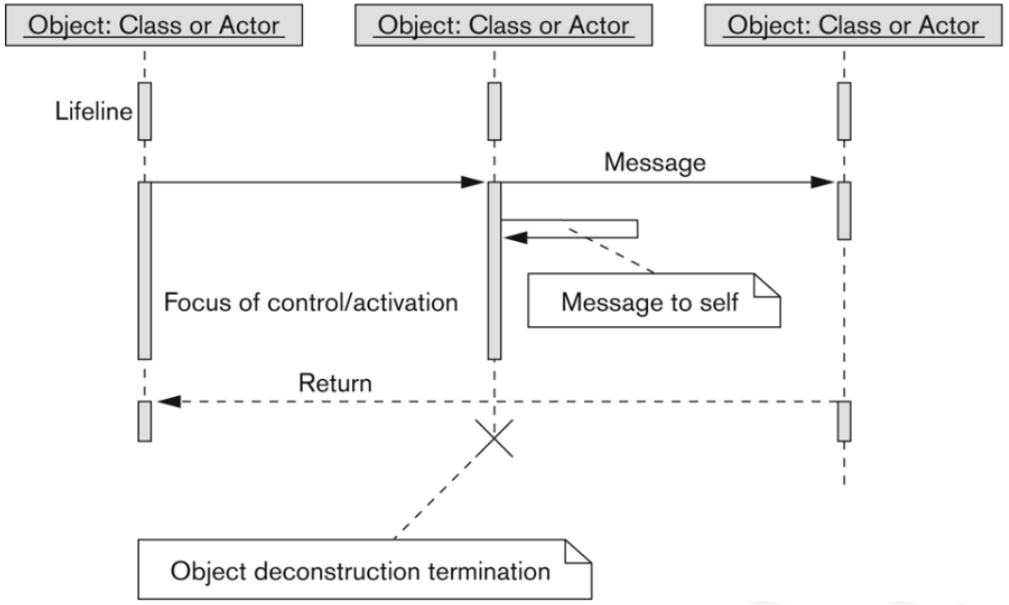
UML Diagrams (contd.)

- **Use Case Diagrams**
  - Model the functional interactions between users and system
  - Describe scenarios of use
  - Serve as a communication tool between users and developers
  - Use case diagram notation shown in figure



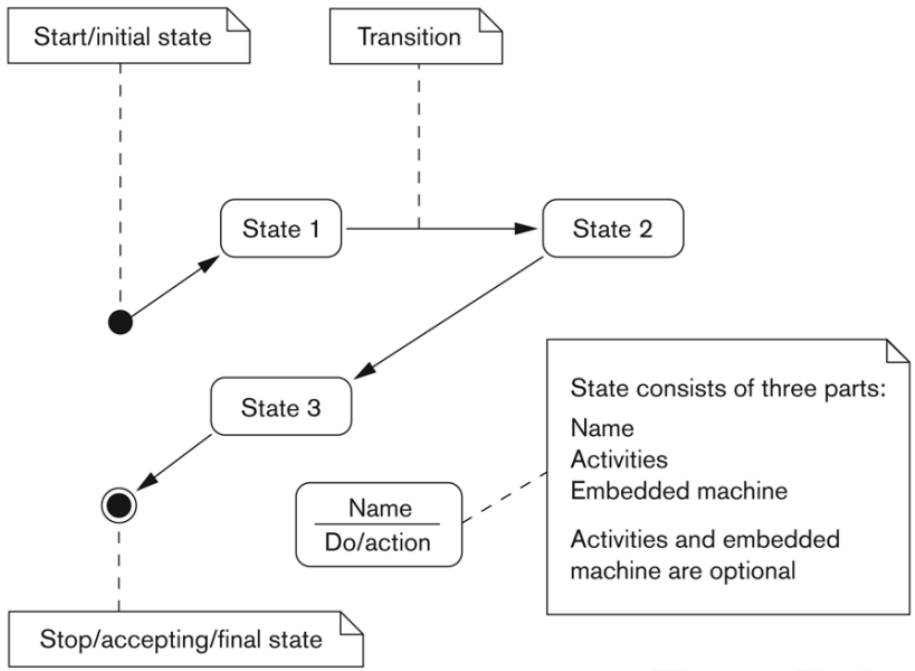
UML Diagrams (contd.)

- **Sequence Diagrams**
  - Represent interactions between various objects over time
  - Relate use cases and class diagrams



UML Diagrams (contd.)

- **Collaboration Diagrams**
  - Represent interactions between objects as a series of sequenced messages
- **Statechart Diagram**
  - Describe how an object's state changes in response to external events
  - Consist of states, transitions, actions, activities and events



UML Diagrams (contd.)

Activity Diagrams

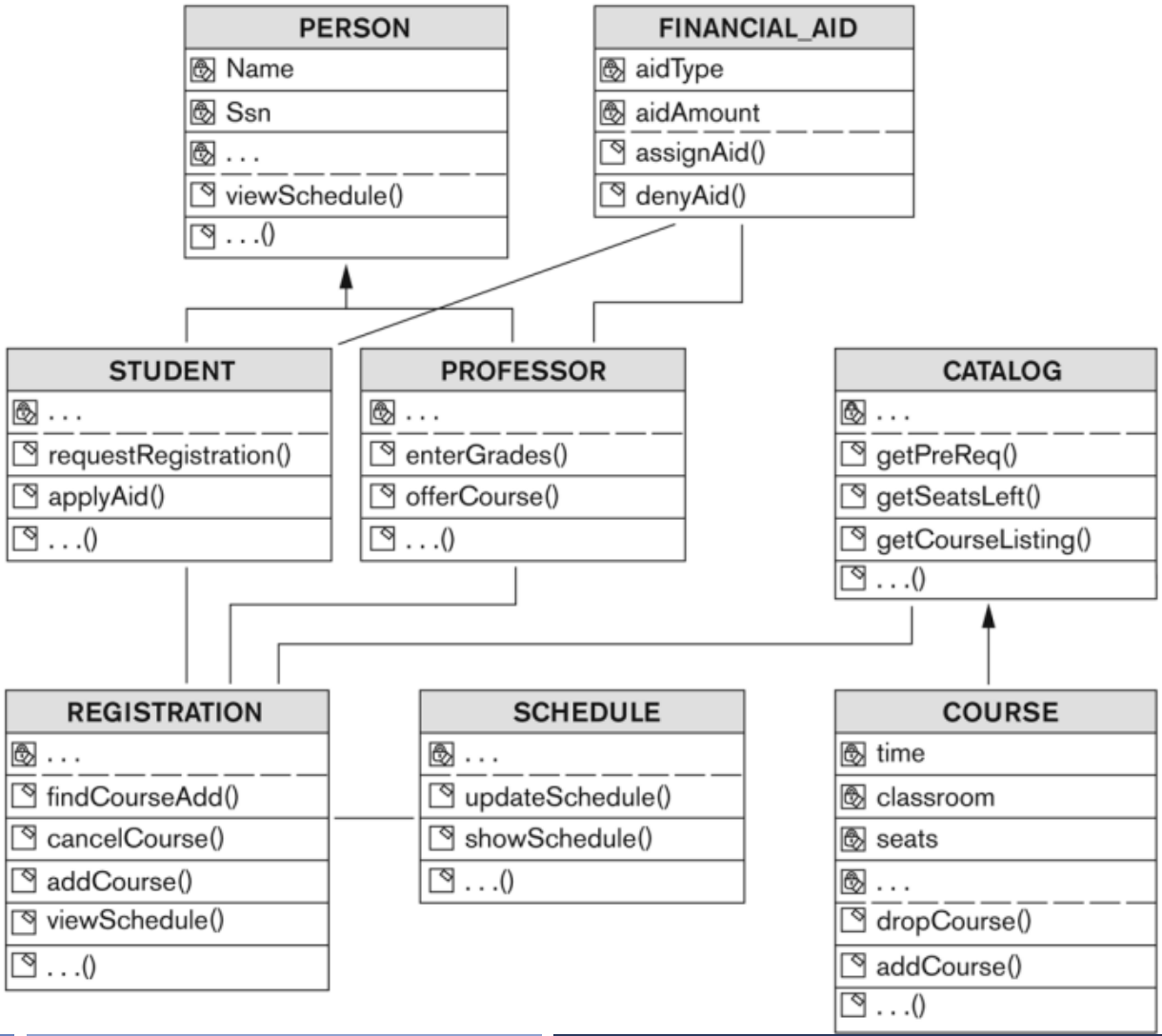
- Model the flow of control from activity to activity
- Flowcharts with states

Salient Features of Rational Rose Data Modeler

- UML based modeling tool for designing databases
- **Reverse Engineering**
  - Generate a conceptual data model from an existing DBMS database or DDL
- **Forward Engineering**
  - Create application/data model
  - Generate DDL from data model

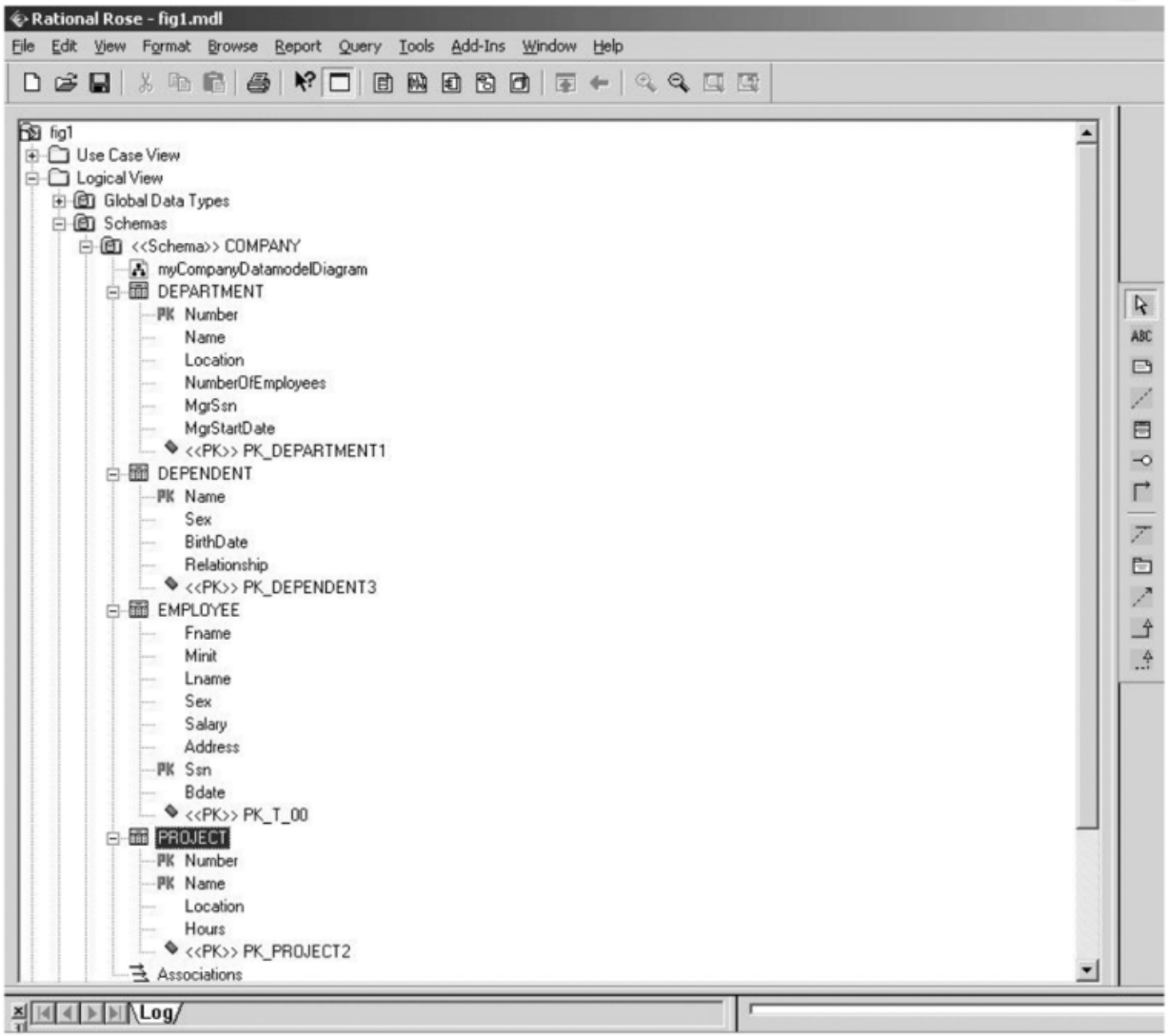
Salient Features of Rational Rose Data Modeler

- The design of the university database as a class diagram



Salient Features of Rational Rose Data Modeler

- Modeling ER diagrams in UML



Salient Features of Rational Rose Data Modeler

- Keeps the data model and database synchronized
  - Gives the option of updating the model or changing the database
- Provides Extensive Domain Support
  - Allows database designers to create a standard set of user-defined types
- Allows Converting between logical and object model design
- Allows easy communication between various developing and design teams
  - Provides a common tool and platform to designers and developers
- Rational Rose Web Publisher

Automated Database Design Tools

Company	Tool	Functionality
Embarcadero Technologies	ER/Studio, DBArtisan	Database modeling in ER and IDEF1x; Database administration
Oracle	Developer 2000, Designer 2000	Database modeling, application development
Persistence Inc.	PowerTier	Mapping from O-O to relational model
Platinum Technology	Platinum ModelMart, ERwin, BPwin	Data, process, and business component modeling
Popkin Software	Telelogic System Architect	Data modeling, object modeling, process modeling
Rational (IBM)	Rational Rose XDE Developer Plus	Modeling in UML and application generation
Resolution Ltd.	XCase	Conceptual modeling up to code maintenance
Sybase	Enterprise Application Suite	Data and business logic modeling
Visio	Visio Enterprise	Data modeling, design and reengineering

Acknowledgments/Contributions

- Some of the images utilised in these slides are subject to copyright
  - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26
  - Yogesh K. Meena, IIT Gandhinagar